

Recursion

Write a recursive function.

1. to calculate the factorial of a number.
2. to find the sum of the first N natural numbers.
3. to print the first N natural numbers in normal order.
4. to print the first N natural numbers in reverse order.
5. to count the number of digits in a positive integer.
6. to calculate the sum of the digits of a positive integer.
7. to compute the power of a number.
8. to compute the product of two numbers without using the multiplication operator.
9. to calculate the nth Fibonacci number.
10. to find the number of ways to climb N steps if you can take 1 or 2 steps at a time.
11. to calculate the sum of all elements in an array.
12. print all elements of an array in normal order.
13. to print all elements of an array in reverse order.
14. to find the maximum element in an array using sequential recursion.
15. to find the minimum element in an array.
16. to check if an array is sorted in ascending order.
17. to search for a value in an array using sequential search.
18. to find the length of a given string.
19. to reverse a string.
20. to check if a string is a palindrome.
21. to count the number of vowels in a string.
22. to count the number of digits in a string.
23. to find the sum of digits in a string.
24. to find whether a word exists inside a sentence or string.

Prepared and Organized by
BADHON NANDI
Student Tutor & FYAT Mentor
Department of Computer Science and Engineering, Brac University

Editing in progress

Email me for any question additions/enquiries: badhon.nandi@g.bracu.ac.bd

Tracing Problems

```
1  public class RecursionTrace1 {
2      public static int sum(int n) {
3          if (n == 0) {
4              return 0;
5          }
6          return n + sum(n - 1);
7      }
8
9      public static void main(String[] args) {
10         int result = sum(5);
11         System.out.println(result);
12     }
13 }
```

```
1  public class RecursionTrace2 {
2      public static int calculate(int n) {
3          if (n <= 0) {
4              return 4;
5          }
6          else if (n % 2 != 0) {
7              return n + calculate(n - 1);
8          }
9          else {
10             return n * calculate(n - 2);
11         }
12     }
13
14     public static void main(String[] args) {
15         int result = calculate(6);
16         System.out.println(result);
17     }
18 }
```

```
1  public class RecursionTrace3 {
2      public static void show(int n) {
3          if (n == 0) {
4              System.out.print("B ");
5              return;
6          }
7
8          System.out.print(n + " ");
9          show(n - 1);
10         System.out.print(n + " ");
11     }
12
13     public static void main(String[] args) {
14         show(3);
15     }
16 }
```

```
1  public class RecursionTrace4 {
2      public static int mystery(int n) {
3          if (n <= 1) {
4              return n + 1;
5          }
6          return mystery(n - 1) + mystery(n - 2);
7      }
8
9      public static void main(String[] args) {
10         int result = mystery(4);
11         System.out.println(result);
12     }
13 }
```

```
1  public class RecursionTrace5 {
2      public static int process(int[] arr, int index) {
3          if (index == arr.length) {
4              return 0;
5          }
6
7          if (arr[index] % 2 == 0) {
8              return arr[index] + process(arr, index + 1);
9          }
10         else {
11             return process(arr, index + 1) - arr[index];
12         }
13     }
14
15     public static void main(String[] args) {
16         int[] nums = {2, 5, 4, 3};
17         int result = process(nums, 0);
18         System.out.println(result);
19     }
20 }
```